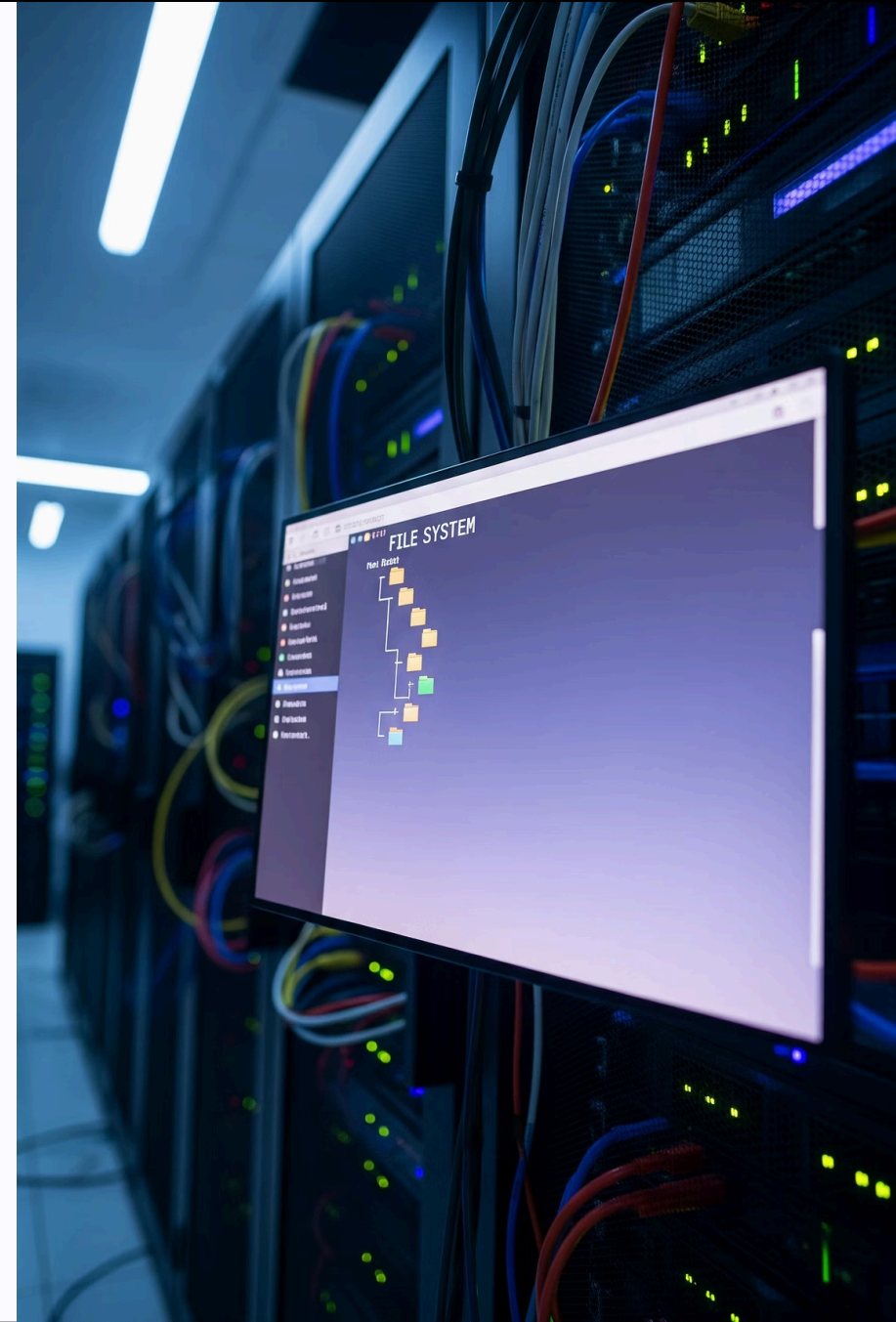


Linux Filesystems, Directories, and Permissions

Essential Knowledge for AI Infrastructure Engineers



Linux Filesystem Basics



Everything is a File

In Linux, almost everything is represented as a file, including hardware devices, processes, and system configurations.



Hierarchical Structure

Filesystem begins at root (/) and branches into standardized directories like /home, /etc, /var, and /usr.



Performance Impact

Your choice of filesystem directly affects **speed, reliability, and scalability** of your AI infrastructure.

Understanding filesystem architecture is fundamental to building robust AI systems.

Common Filesystem Types

ext4

Default for most Linux distributions. Reliable with journaling to prevent corruption after crashes.

XFS

Optimized for **high-performance servers** and parallel I/O.
Scales to massive file sizes.

NTFS/FAT32

Used primarily for Windows compatibility in dual-boot or external drive scenarios.

NFS

Network File System for shared storage across compute clusters in distributed training.

Object Stores

S3, GCS, Azure Blob often mounted as filesystems for large-scale ML datasets.



Key Directories in AI Workloads

Understanding the purpose of standard directories helps organize ML projects efficiently.

`/home`

User directories containing personal files, code repositories, and configuration files.

Example: `/home/data-scientist/projects/nlp-model/`

`/data (custom)`

Non-standard directory commonly created specifically for **large ML datasets**, often on dedicated volumes.

Example: `/data/imagenet/training/`

`/var`

System logs, cache files, and temporary data. Critical for debugging model training issues.

Example: `/var/log/nvidia-smi.log`

`/etc`

System-wide configurations for ML frameworks, CUDA, and other dependencies.

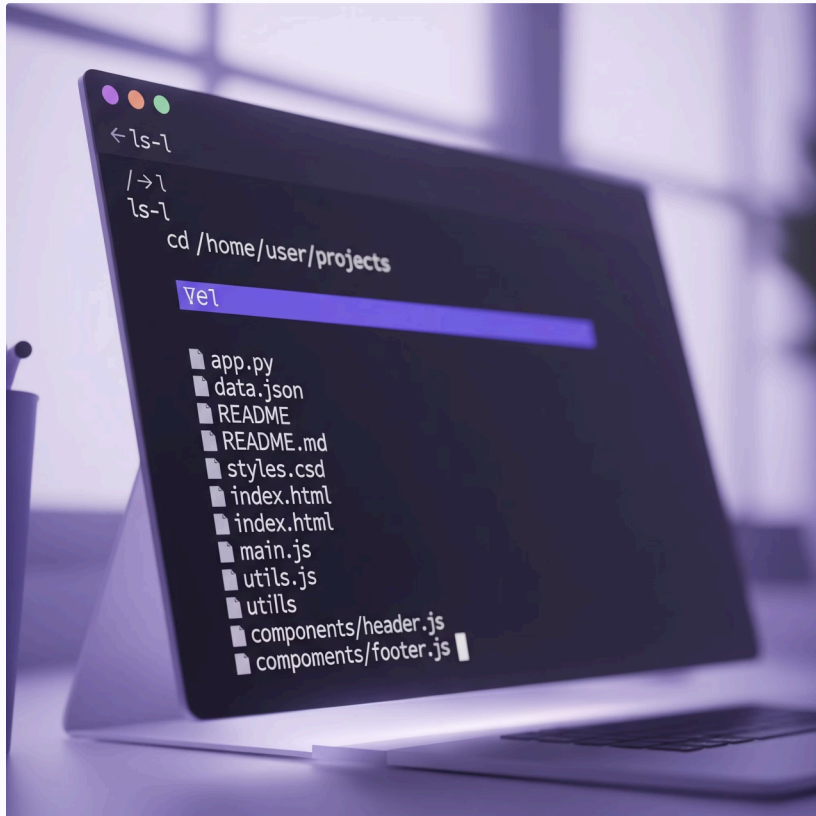
Example: `/etc/jupyter/jupyter_notebook_config.py`

`/mnt` or `/media`

Mount points for external storage volumes, NFS shares, or cloud object stores.

Example: `/mnt/s3-datasets/`

Navigating Directories



Essential Commands

- `pwd` - Print working directory (current location)
- `cd /path/to/folder` - Change to specified directory
- `ls -lh` - List files with human-readable sizes
- `tree` - Visualize directory hierarchy (may need installation)

Best Practices

Organize your AI projects with consistent directory structures:

```
project/  
├── data/ # Raw and processed datasets  
├── models/ # Saved model checkpoints  
├── notebooks/ # Jupyter notebooks  
├── src/ # Source code  
└── configs/ # Configuration files
```

File Ownership

In multi-user AI environments, file ownership determines who can access and modify data.

1

Owner

The user who created the file or was assigned ownership. Has primary control over the file.

2

Group

Collection of users who share the same permissions to the file (e.g., data-scientists, ml-engineers).

View ownership with `ls -l`:

```
-rw-r--r-- 1 jsmith ml-team 1.2G May 15 14:30 training-data.h5
```

Change ownership with:

```
chown user:group filename
```



Linux Permission Model



User (u)

Permissions for the file owner



Group (g)

Permissions for members of the file's group



Others (o)

Permissions for everyone else

Read (r)

View file contents or list directory contents

Write (w)

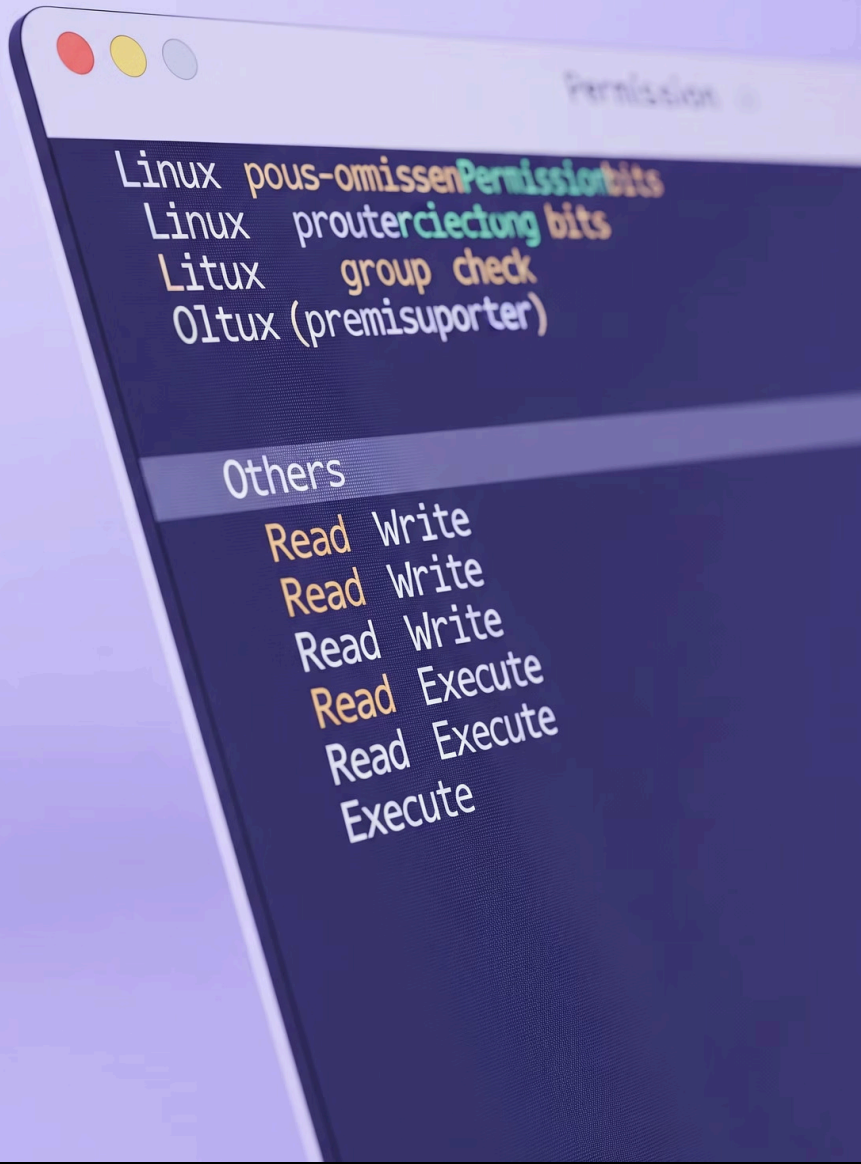
Modify file or create/delete files in directory

Execute (x)

Run file as program or access directory

Example: `-rwxr-xr--` means:

- Owner can read, write, execute
- Group can read and execute
- Others can only read



Practical Permission Scenarios

Restrict Dataset Access

```
chmod 750 /data/medical-images/
```

Ensures only project team members can access sensitive training data.

Shared Model Weights

```
chmod 644 /models/pretrained/*.h5
```

Everyone can read models, only owner can modify.

Make Scripts Executable

```
chmod +x train.sh
```

Allows script to be executed directly (./train.sh).

⊗ Security Warning

Never use `chmod 777` (all permissions for everyone)!

This creates serious security vulnerabilities in your infrastructure.

Common Permission Patterns

- `chmod 755` - Executable scripts/programs
- `chmod 644` - Configuration files
- `chmod 700` - Private user files
- `chmod 770` - Team-only directories

Why Permissions Matter for AI Infrastructure

Proper filesystem permissions are foundational to secure, reliable AI systems.

Data Security
Protects **sensitive datasets & credentials** from unauthorized access

Prevent Accidents
Stops inadvertent deletion or corruption of critical model files



Multi-user Clusters

Essential for shared GPU resources with multiple teams

Compliance

Supports HIPAA, GDPR, SOC2 requirements for data handling

Mastering filesystem permissions is a core competency for AI infrastructure engineers.